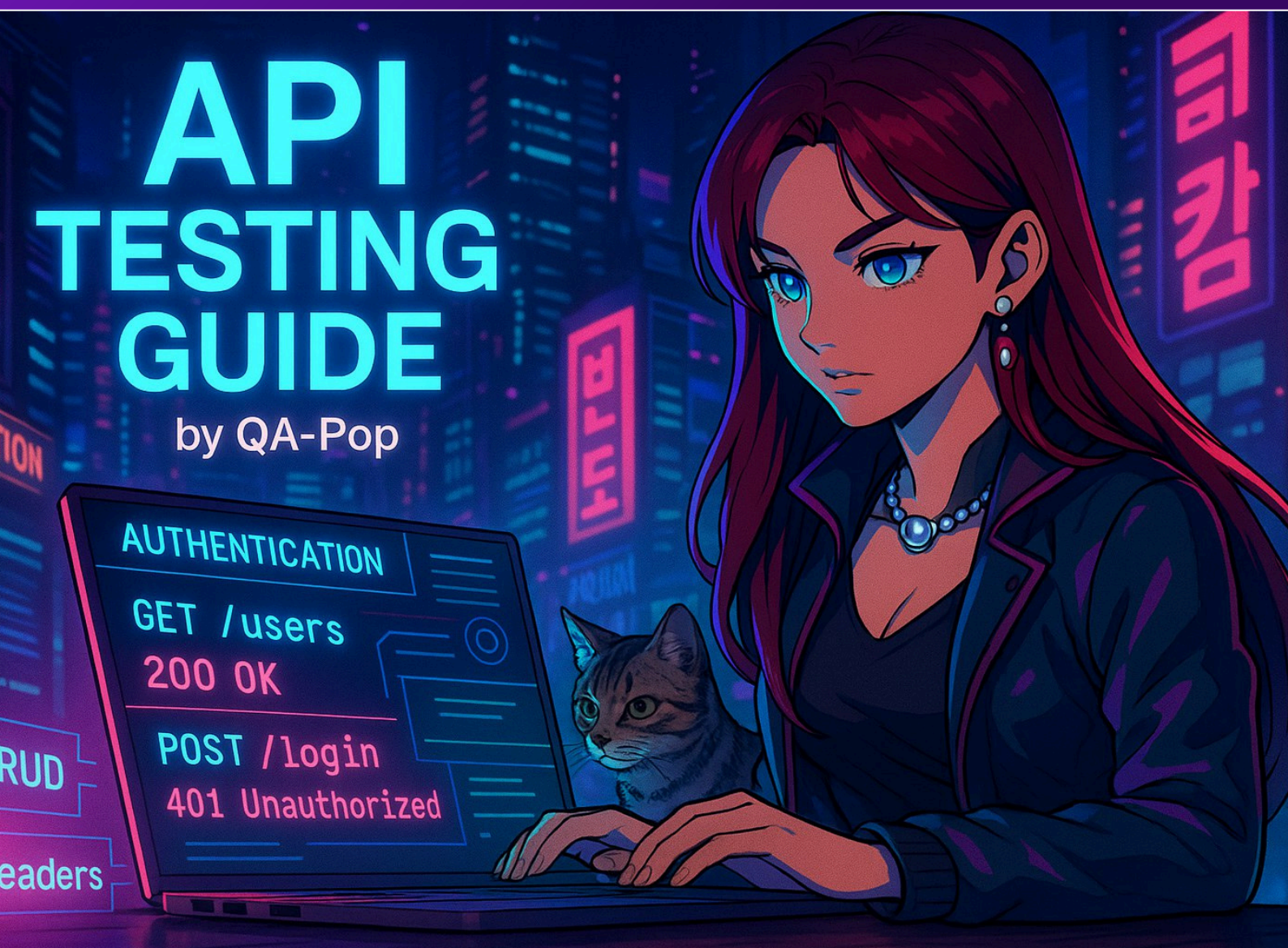


API- ТЕСТИРОВАНИЕ

API TESTING GUIDE

by QA-Pop



Привет!

Привет! Меня зовут Алена, и я уже пятый год тестирую веб-приложения.

API-тестирование — одна из самых интересных частей моей работы, и в этом гайде я собрала все свои знания, чтобы помочь вам разобраться в теме.

ЭТОТ ГАЙД ДЛЯ ВАС, ЕСЛИ:

- Вы только начинаете тестировать API и хотите *понять основы*;
- Вам нужно *быстро повторить ключевые моменты* перед собеседованием;
- *Есть пробелы в знаниях*, и вы хотите закрыть их раз и навсегда.

У меня есть телеграм канал *Алена QA_POP* — там много полезных материалов по тестированию, советы и разборы. [ссылка тут](#)
Присоединяйтесь!

Alena

Содержание

ЧТО ТАКОЕ API ТЕСТИРОВАНИЕ?	03
ОСНОВНЫЕ ТИПЫ ТЕСТИРОВАНИЯ API	06
КАК КОДЫ HTTP ПОМОГАЮТ БЫСТРО ДИАГНОСТИРОВАТЬ ПРОБЛЕМЫ В API?	11
МЕТОДЫ HTTP-ЗАПРОСОВ В API И ИХ ОТЛИЧИЯ	14
JSON И XML — ЧТО ЭТО ТАКОЕ?	17
ИНСТРУМЕНТЫ ДЛЯ ТЕСТИРОВАНИЯ API	19





Что такое API-тестирование?

За последние 5 лет API-тестирование из "желательного" навыка превратилось в must-have и теперь встречается в большинстве вакансий для тестировщиков, особенно для позиций уровня Middle и выше. Вот несколько причин почему этот навык стал таким популярным:

ЭКОНОМИЯ РЕСУРСОВ:

Ранняя проверка API позволяет тестировать бизнес-логику и находить баги до готовности UI.

Скорость проверок и диагностики: API-тесты выполняются в несколько раз быстрее и поддерживать их легче, чем UI (авто-)тесты.

АРХИТЕКТУРНЫЙ СДВИГ:

Микросервисы заменили монолиты: современные системы состоят из десятков компонентов, взаимодействующих по API.

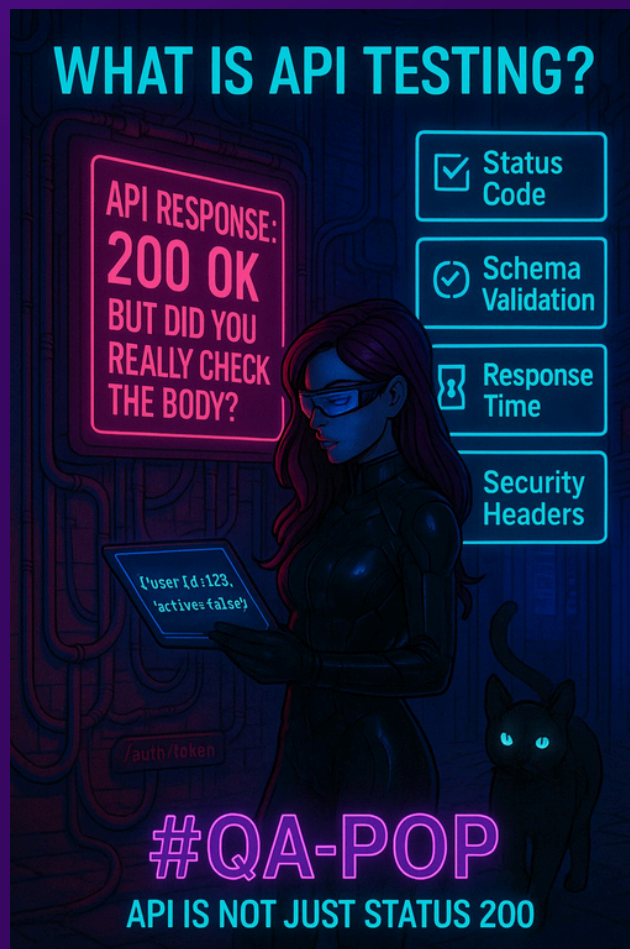
Отделение фронтенда от бэкенда: 90% логики теперь реализовано в API.

API тестирование — это тип тестирования программного обеспечения, который проверяет Application Programming Interface (API) на соответствие требованиям по функциональности, надежности, производительности и безопасности.

В отличие от UI-тестирования, API-тестирование работает на сервисном уровне и проверяет взаимодействие между системами или компонентами без графического интерфейса.

Цели API-тестирования:

- 01 Проверка корректности работы API (возвращает ли он корректные данные для валидных / невалидных запросов)
- 02 Оценка производительности (скорость ответа, устойчивость к нагрузке)
- 03 Совместимость с другими системами (интеграции с внешними сервисами, например, с платежными системами)
- 04 Оценка уязвимости данных (безопасность, предотвращение утечек данных, DDoS-атак)



Что именно тестируется в API?

Эндпоинты (URL)

HTTP-методы — GET, POST, PUT, DELETE и др.

Параметры запроса — Query, Path, Headers, Body.

Коды статусов — 200 (OK), 400 (Bad Request), 500 (Server Error) и др.

Формат данных — JSON, XML, правильность структуры ответа.

Авторизация — Работа с токенами (JWT, OAuth), API-ключами.

Обработка ошибок — Как API реагирует на неверные данные.

ПРОСТЫЕ ПРИМЕРЫ API-ТЕСТОВ НА КОТИКАХ:

Тест поиска котиков по породе (GET)

GET https://api.thecatapi.com/v1/images/search?breed_ids=beng

Что проверяем:

Возвращаются только бенгальские кошки

У каждой картинки есть `breed_id = "beng"`

Тест загрузки котика (POST)

POST <https://api.thecatapi.com/v1/images/upload>

Тело запроса (multipart/form-data):

file: ваш_кот.jpg

sub_id: ваш_идентификатор

Что проверяем:

Статус 201 Created

В ответе есть ID и url

загруженного изображения

Тест добавления в избранное (POST)

POST <https://api.thecatapi.com/v1/favourites>

Тело запроса:

```
{
  "image_id": "2QluO0f9H",
  "sub_id": "crying_cat_1"
}
```

Что проверяем:

Статус 200 OK

В ответе есть field "id"



Основные типы тестирования API, которые важно знать :

1. Тестирование безопасности (Security Testing)

Цель: Выявить уязвимости и защитить конфиденциальные данные.

Что тестируется:

Авторизация/аутентификация (OAuth, JWT, API-ключи).

Защита от инъекций (SQL, XSS).

CSRF-токены и CORS-политики.

Rate limiting (ограничение запросов).

 **Примеры:**

Попытка доступа к API без токена → 401 Unauthorized.

Проверка, что конфиденциальная информация пользователя передается безопасно (например, не в plain text виде).

2. Валидационное тестирование (Validation testing)

Цель: Проверить, что реализованный API соответствует заявленным функциональным и бизнес-требованиям при валидных входных данных (часть functional testing).

Что тестируется:

Соответствие бизнес логике при заданных входных данных.

Соответствие структуры ответов заявленным стандартам (JSON/XML).

Корректность типов данных, обязательных полей и вложенных структур.

 **Пример:**

Создание валидного пользователя со всеми корректными атрибутами → 201 Created.

3. UI testing (взаимодействие пользовательского интерфейса и API)

Цель: Проверить насколько корректно фронтенд (UI) и бэкенд (API) работают вместе.

Что тестируется:

Проверка отправки данных из UI в API (вызов правильного API-запроса).

Проверка отображения данных из API в UI и синхронизации данных.

Проверка обработки ошибок API в UI.

Валидация нагрузочных элементов (спиннеры, сообщения о загрузке).

 Пример:

При регистрации UI отправляет корректные данные и запрос идет на правильный эндпоинт.

4. Функциональное тестирование (Functional testing)

Цель: Проверить, что API работает согласно спецификации и выполняет все необходимые операции.

Что тестируется:

Валидные входные данные.

Работа методов (GET, POST, PUT, DELETE и др.).

Обработка ошибок (коды 400, 404, 500 и т.д.).

Валидация входных/выходных данных (JSON/XML-схемы).

 Пример:

Отправка POST-запроса на создание пользователя и проверка, что он появился в БД.

5. Error Testing (тестирование ошибок)

Цель: Проверить, что API обрабатывает некорректные запросы и сохраняет стабильность системы при сбоях.

Что тестируется:

Невалидные входные данные.

Обработка отсутствующих ресурсов.

Обработка ошибок авторизации/доступа и ошибок сервера.

 Пример:

Запросы к несуществующим URL → 404 Not Found.

6. Пентест (Penetration Testing)

Цель: Целенаправленная атака на API для поиска уязвимостей (часть security testing).

Что тестируется:

Инъекции (SQL, NoSQL).

Конфиденциальные данные (например, подбор токенов/паролей).

Уязвимости бизнес-логики (например, отрицательные суммы в платежах).

💡 **Пример:**

Подделка JWT-токена → 403 Forbidden

7. Interoperability Testing (Тестирование взаимодействия)

Цель: проверка способности системы корректно взаимодействовать с другими системами, компонентами или устройствами.

Что тестируется:

Интеграция с внешними сервисами (платежные системы, геосервисы, соцсети).

Кросс-платформенные проверки: ОС (Windows, Linux, macOS), браузеры (Chrome, Firefox, Safari), мобилки (iOS, Android).

Совместимость форматов данных и версий API.

💡 **Пример:**

После оплаты через ХБанк статус заказа в системе меняется на «Оплачено».



8. Нагрузочное тестирование (Load Testing)

Цель: Проверить устойчивость и масштабируемость API при высоких нагрузках.

Что тестируется:

Время отклика при увеличении числа запросов.

Устойчивость к высоким нагрузкам (RPS — запросов в секунду).

Пределы пропускной способности.

Поведение при достижении лимитов (Rate Limiting).

💡 **Пример:**

10000 одновременных GET-запросов к /products → API не должно «падать».

9. Fuzz Testing (Фаззинг-тестирование)

Цель: проверка работы с невалидными/случайными данными с целью выявления уязвимостей и сбоев (часть error testing).

Что тестируется:

Обработка нестандартных параметров запросов.

Обнаружение скрытых уязвимостей (SQL-запросы, отправка битых TCP-пакетов).

Поведение при неожиданных типах данных (например, передача строки вместо числа).

💡 **Пример:**

Загрузка "битого" PNG-файла → выполнение произвольного кода.



Структура HTTP: Request и Response

HTTP (HyperText Transfer Protocol) — это протокол передачи данных в интернете, который позволяет браузерам и серверам обмениваться информацией.

КАК РАБОТАЕТ?

- 01 Клиент (браузер, приложение) отправляет HTTP-запрос на сервер
- 02 Сервер обрабатывает запрос и отправляет обратно HTTP-ответ
- 03 Клиент получает ответ (например, загружает страницу или данные)

Характеристика	HTTP Request (Запрос)	HTTP Response (Ответ)
Назначение	Запрос данных/действия от сервера	Ответ сервера на запрос
Кто отправляет	Клиент (браузер, приложение)	Сервер
Структура	1. Стартовая строка 2. Заголовки 3. Тело (опционально)	1. Статусная строка 2. Заголовки 3. Тело (опционально)
Пример	GET /index.html HTTP/1.1 Host: example.com	HTTP/1.1 200 OK Content-Type: text/html <html>...</html>
Ключевые элементы	Методы: GET, POST, PUT, DELETE	Коды статуса: 200, 404, 500
Когда используется	При загрузке страницы, отправке формы	При ответе сервера на запрос

Как коды HTTP помогают быстро диагностировать проблемы в API?

HTTP-коды — это первый индикатор состояния API, который помогает быстро понять, успешно ли выполнен запрос или возникла ошибка.

ВОТ КАК ОНИ УСКОРЯЮТ ДИАГНОСТИКУ ПРОБЛЕМ

1. Мгновенное понимание типа ошибки

Коды делятся на 5 категорий, что сразу сужает круг возможных проблем:

1xx (Информационные) — запрос в процессе.

2xx (Успех) — всё ок, можно анализировать тело ответа.

3xx (Перенаправление) — нужно выполнить дополнительное действие (например, перейти по новому URL).

4xx (Ошибка клиента) — проблема на стороне запроса (неверные данные, нет прав и т.д.).

5xx (Ошибка сервера) — проблема на стороне сервера (например, база данных недоступна).

Пример:

Получили 404? Значит, ресурс не существует (проверяем URL или ID).

Получили 500? Значит, сервер сломался (нужно смотреть логи сервера).

2. Точечная диагностика без анализа тела ответа

Иногда тело ответа пустое или содержит минимум данных, но код уже дает ключевую информацию:

401 Unauthorized → Требуется авторизация (нет токена или он неверный).

403 Forbidden → Нет доступа (есть токен, но прав недостаточно).

429 Too Many Requests → Превышен лимит запросов (нужно ждать или уменьшить нагрузку).

Пример:

HTTP/1.1 403 Forbidden

Даже без тела ответа ясно: у клиента нет прав на операцию.

3. Стандартизация ошибок

HTTP-коды универсальны для всех API, поэтому:

Разработчики сразу понимают их смысл без чтения документации.

Инструменты (Postman, Swagger, мониторинговые системы) автоматически классифицируют ответы.

Пример:

В Postman 4xx/5xx подсвечиваются красным, а 2xx — зеленым.

4. Упрощение логирования и мониторинга

Системы мониторинга (Prometheus, Grafana, ELK) агрегируют ошибки по кодам:

Можно быстро найти «топ-5 ошибок 500 за последний час».

Настроить алерты при росте 5xx (проблемы сервера) или 429 (риск DDoS).

Пример дашборда:

500 Internal Server Error — 100 запросов

400 Bad Request — 30 запросов

200 OK — 1000 запросов



5. Оптимизация обработки ошибок на клиенте

Клиент (браузер, мобильное приложение) может адаптировать поведение на основе кода:

301/302 → автоматически перейти по новому URL.

401 → показать окно входа.

503 → вывести сообщение «Сервис временно недоступен».

6. Интеграция с инструментами тестирования

Автотесты API могут проверять коды для валидации сценариев:

Ожидаем 200 при корректном запросе.

Ожидаем 400 при отправке невалидных данных.

HTTP-коды — это универсальный язык общения между клиентом и сервером, который:

- 01 Позволяют мгновенно определить тип проблемы;
- 02 Снижают время на анализ ошибок (достаточно взглянуть на код);
- 03 Стандартизируют логирование, мониторинг и тестирование.

Методы HTTP-запросов в API и их отличия

HTTP-методы определяют действие, которое нужно выполнить с ресурсом. Они играют ключевую роль в RESTful API и других web-интерфейсах.

ОСНОВНЫЕ МЕТОДЫ:

1

GET — ПОЛУЧЕНИЕ ДАННЫХ

Назначение: Запрашивает данные с сервера (без изменения состояния).

Идемпотентность: Да (повторный запрос не меняет результат).

Пример:

GET /api/users/1 HTTP/1.1 → Возвращает данные пользователя с id=1.

2

POST — СОЗДАНИЕ ДАННЫХ

Назначение: Отправляет данные на сервер для создания ресурса.

Идемпотентность: Нет (повторный запрос может создать новую запись с теми же данными).

Пример:

POST /api/users HTTP/1.1 {"name": "Alice", "age": 25} → Создает нового пользователя.

3

PUT — ПОЛНОЕ ОБНОВЛЕНИЕ РЕСУРСА

Назначение: Заменяет **весь** ресурс новыми данными.

Идемпотентность: Да (повторный запрос не меняет результат).

Пример:

PUT /api/users/1 HTTP/1.1 {"name": "Bob", "age": 30} → Полностью перезаписывает данные пользователя id=1.

4

PATCH — ЧАСТИЧНОЕ ОБНОВЛЕНИЕ

Назначение: Обновляет **только указанные поля** ресурса.

Идемпотентность: Зависит от реализации (обычно да).

Пример:

PATCH /api/users/1 HTTP/1.1 {"age": 31} → Меняет только возраст пользователя id=1.

5

DELETE — УДАЛЕНИЕ РЕСУРСА**Назначение:** Удаляет указанный ресурс.**Идемпотентность:** Да (повторные запросы не изменяют состояние).**Пример:**

DELETE /api/users/1 HTTP/1.1 → Удаляет пользователя id=1.

6

HEAD — ПОЛУЧЕНИЕ ЗАГОЛОВКОВ (БЕЗ ТЕЛА)**Назначение:** Как GET, но возвращает только HTTP-заголовки (без тела).**Пример:** Проверка существования ресурса перед загрузкой.

7

OPTIONS — ИНФОРМАЦИЯ О ДОСТУПНЫХ МЕТОДАХ**Назначение:** Возвращает список поддерживаемых методов для ресурса.**Пример:**

OPTIONS /api/users HTTP/1.1 → Может вернуть: HTTP/1.1 200 OK Allow: GET, POST, OPTIONS

КЛЮЧЕВЫЕ ОТЛИЧИЯ:

Метод	Изменяет данные?	Идемпотентность*	Пример использования
GET	✗ Нет	✓ Да	Получение списка пользователей
POST	✓ Да	✗ Нет	Создание нового пользователя
PUT	✓ Да	✓ Да	Полная замена данных
PATCH	✓ Да	⚠ Зависит	Частичное обновление данных
DELETE	✓ Да	✓ Да	Удаление пользователя

Идемпотентность метода — это свойство метода, при котором его многократное выполнение даёт тот же результат, что и однократное.

ПРИМЕРЫ:

Кнопка лифта "Заккрыть дверь":

Нажатие один раз закрывает дверь.

Повторные нажатия ничего не меняют — дверь уже закрыта.

Голосование за кандидата

Действие: Отдать голос за одного кандидата (если нельзя голосовать дважды).

Повторный голос не изменит результат — учтён только первый.

КОГДА КАКИЕ МЕТОДЫ ИСПОЛЬЗОВАТЬ?

GET — Получение данных (списков, деталей объекта).

POST — Создание нового ресурса (пользователь, заказ).

PUT — Полное обновление (например, замена профиля пользователя).

PATCH — Частичное обновление (например, смена email или статуса).

DELETE — Удаление ресурса (пользователя, товара).



JSON и XML — что это такое?

Представьте, что два компьютера хотят обмениваться данными, но они не могут говорить по-человечески, как мы. Им нужен **удобный формат**, чтобы понимать друг друга.

Тут на помощь приходят **JSON и XML** — это «языки» для *передачи данных*.

JSON (JAVASCRIPT OBJECT NOTATION) КАК ВЫГЛЯДИТ?

```
{  
  "cat": {  
    "name": "Куся",  
    "age": 10,  
    "colors": ["рыжий", "белый", "черный"],  
    "isFluffy": true,  
    "owner": null  
  }  
}
```

Разбираем структуру:

`{}` — объект (в данном случае — описание кота).

"ключ": "значение" — пары данных (например, "name": "Барсик").

`[]` — массив данных (список значений, например, цвета кота).



Типы данных: Строки ("Барсик"),
Числа (2),
Булевы значения (true/false),
null (пустое значение).

P.S. Если хотите потренироваться, попробуйте описать в JSON своего питомца (или воображаемого котика).

XML (EXTENSIBLE MARKUP LANGUAGE) КАК ВЫГЛЯДИТ?

```
<cat>
  <name>Куся</name>
  <age>10</age>
  <colors>
    <color>черный</color>
    <color>белый</color>
  </colors>
  <isFluffy>true</isFluffy>
  <owner></owner>
</cat>
```

```
<cat name="Куся" breed="уличная" age="10">
  <favoriteFood>рыба</favoriteFood>
  <favoriteFood>вкусняшки</favoriteFood>
  <vaccinated>yes</vaccinated>
</cat>
```

Разбираем структуру:

Теги (<tag>) — как "контейнеры" для данных.

Закрывающий тег (</tag>) обязателен.

Можно добавлять атрибуты (<cat breed="британский">).

Пустые теги можно писать сокращенно (<owner/>).

JSON VS XML: ГЛАВНЫЕ ОТЛИЧИЯ

Характеристика	JSON	XML
Читаемость	Проще (как текст)	Сложнее (много тегов)
Размер	Компактный	Громоздкий
Скорость	Быстрее обрабатывается	Медленнее
Где используют	Современные API (REST)	Старые системы (SOAP)

ПРИМЕРЫ:

JSON — если делаете мобильное приложение или веб-API.

XML — если работаете с корпоративными системами (банки, 1С).

Инструменты для тестирования API:

1

GUI-ИНСТРУМЕНТЫ (ВИЗУАЛЬНЫЕ ПРОГРАММЫ)

Postman - популярный инструмент с удобным интерфейсом, в котором можно сохранять коллекции запросов, писать автотесты на **JavaScript**.

Пример:

Отправка **GET**-запроса: GET https://cats/1

2

SWAGGER (OPENAPI) - ИСПОЛЬЗУЕТСЯ ДЛЯ ДОКУМЕНТИРОВАНИЯ И ТЕСТИРОВАНИЯ API

Пример:

Заходим на swagger-ui (например, <http://petstore.swagger.io/>).

Выбираем эндпоинт, например, GET /pet/{petId}.

Вводим petId = 1, нажимаем **Execute** — получаем ответ.



3

НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ (ПРОВЕРКА ПРОИЗВОДИТЕЛЬНОСТИ)

JMeter — инструмент для нагрузочного тестирования и автоматизации тестов.

Пример:

Проверка, как API поведёт себя при наплыве пользователей.

Проверка с целью поиска "узких мест" (медленные запросы, ошибки под нагрузкой).

4

КОМАНДНЫЕ УТИЛИТЫ ДЛЯ ТЕСТИРОВАНИЯ API (РАБОТА ЧЕРЕЗ ТЕРМИНАЛ)

cURL (Client URL) — позволяет отправлять HTTP-запросы к API прямо из командной строки (терминала) без графического интерфейса.

Пример:

GET-запрос (получить данные)

```
curl -X GET "https://cats.com/posts/1"
```

